

酒店 Agent 数据闭环改造说明

文档日期：2026-09-21

适用项目：HelloAgents 智能旅行助手

改造范围：酒店 Agent、高德 POI Service、规划 Agent、数据模型与前端类型

当前状态：酒店身份及核心 POI 字段已经形成可信闭环；价格、评分、距离和费用仍属于估算信息

1. 改造背景

改造前，酒店 Agent 自己调用高德工具，并将工具结果或自然语言结果直接交给规划 Agent。酒店事实数据与模型生成内容混在一起，规划 Agent 可能修改酒店名称、地址和坐标，或者生成候选列表中不存在的酒店。

本次改造重新划分了职责：

高德 Service 提供真实酒店候选，酒店 Agent 只负责筛选，规划 Agent 只负责安排，Python 代码负责结构校验、白名单校验和可信字段回填。

2. 改造后的数据链路



职责边界如下：

数据或决策	负责人
POI ID、名称、地址、类型、坐标	高德 Service
是否符合住宿偏好、推荐理由	酒店 Agent
每天安排哪家候选酒店	规划 Agent
结构、数量、重复、归属检查	Python 代码

3. 酒店 Agent 职责调整

酒店 Agent 不再持有或调用高德 MCP 工具。程序先通过 `AmapService.search_poi()` 获取真实候选，再把候选酒店与已选景点一起传给酒店 Agent。

酒店 Agent 只能输出：

```
{
  "hotels": [
    {
      "poi_id": "候选列表中的真实POI ID",
      "reason": "推荐理由"
    }
  ]
}
```

它不能创建或修改 POI ID，也不负责生成酒店名称、地址、类型和坐标。

4. 结构化输出模型

项目新增两个 Pydantic 模型：

- `RankedHotel`：保存酒店 POI ID 和推荐理由；
- `HotelSelection`：保存酒店 Agent 选中的酒店列表。

最终展示模型 `Hotel` 增加必填的 `poi_id`，前端 TypeScript `Hotel` 接口同步增加该字段，保证前后端契约一致。

5. 酒店候选查询

住宿偏好会转换为酒店搜索关键词。已经以“酒店”或“宾馆”结尾时不再重复追加“酒店”，避免出现“经济型酒店酒店”之类的查询。

酒店候选搜索失败会转换为 `ExternalServiceError`。API 层将其映射为 HTTP 503，前端不会把外部服务失败误认为成功行程。

6. 酒店 Agent 输入

`_build_hotel_query()` 接收：

- 原始旅行请求；

- 高德返回的酒店候选 `List[POIInfo]`；
- 景点 Agent 已选景点对应的 `List[POIInfo]`。

传入已选景点是为了让酒店 Agent 能根据主要游览区域判断住宿位置，而不是只依据酒店名称筛选。

7. 第一层白名单：校验酒店 Agent

酒店 Agent 返回结果后，程序依次执行：

1. 从响应中提取 JSON；
2. 验证返回值必须是 JSON 对象；
3. 使用 `HotelSelection` 验证结构；
4. 验证选择数量必须为 1 至 3 家；
5. 拒绝重复 POI ID；
6. 拒绝候选列表之外的 POI ID。

非法 JSON 和结构错误转换为 `AgentOutputError`，白名单或数量错误转换为 `PlanValidationError`。

8. 可信酒店数据

通过第一层校验后，程序按照选中的 POI ID 从候选索引中构造 `trusted_hotels`：

```
poi_id    ← 高德 POI
name      ← 高德 POI
address   ← 高德 POI
location  ← 高德 POI
type      ← 高德 POI
reason    ← 酒店 Agent
```

规划 Agent 只接收这份可信酒店列表，不再接收酒店 Agent 的原始自然语言输出。

9. 第二层白名单：校验规划 Agent

规划 Agent 的提示词要求每天的酒店必须包含 `poi_id`，并且只能使用输入酒店列表中的 POI ID。

提示词不能代替程序校验，因此最终 `TripPlan` 解析后还会检查：

- 每天必须包含酒店；
- 酒店必须包含 POI ID；
- POI ID 必须属于酒店 Agent 的选中集合。

即使规划 Agent 生成了候选外酒店，结果也不会返回给用户。

10. 最终可信字段回填

通过第二层白名单后，代码使用高德候选数据覆盖规划 Agent 输出的以下字段：

- 酒店名称；
- 酒店地址；
- 经纬度坐标；

- 酒店类型。

因此，即使规划 Agent 修改这些字段，最终响应仍以高德 POI 详情为准。

同一家酒店可以在多天行程中重复入住。酒店重复入住是正常业务行为，不使用景点的跨天去重规则。

11. 前端与端到端调整

前端 `Hotel` 类型增加 `poi_id`。多 Agent 串行执行可能超过原来的两分钟请求上限，因此 Axios 超时时间调整为十分钟。

端到端测试应确认：

- 请求成功后页面跳转到 `/result`；
- 每家酒店都有真实 POI ID；
- 名称、地址、坐标和类型来自高德候选；
- 酒店卡片和地图正常显示；
- 外部服务失败时返回 503；
- Agent 输出无效时返回结构化业务错误。

12. 当前限制

12.1 价格和评分不是可信事实

当前高德 POI 适配模型没有提供稳定的实时房价和评分来源，因此 `price_range`、`rating` 和 `estimated_cost` 仍可能由规划 Agent 估算。它们不能被解释为实时预订价格。

12.2 距离仍是估算信息

酒店到景点的 `distance` 尚未接入真实路线数据。后续路线模块应根据酒店和景点坐标调用高德路线服务计算距离与耗时。

12.3 候选数量受 Service 限制

`AmapService.search_poi()` 当前最多解析前十个搜索结果，并为每个结果查询详情。候选质量依赖高德搜索排序和住宿关键词。

12.4 自动化测试仍需扩充

现有通用解析和规划失败语义测试可以验证基础设施，但酒店空选择、重复 ID、候选外 ID、可信字段回填等场景仍适合增加独立单元测试。

13. 涉及文件

```
backend/app/agents/trip_planner_agent.py
backend/app/models/schemas.py
frontend/src/services/api.ts
frontend/src/types/index.ts
docs/hotel-agent-data-closure-guide.md
CHANGELOG.md
```

14. 核心总结

酒店模块已经从“模型搜索并整理酒店”调整为“程序获取事实、Agent 做选择、程序执行校验和回填”。

本次改造最重要的结果不是增加了更多提示词，而是建立了两层白名单和事实字段回填机制，使规划结果中的酒店身份与核心 POI 信息可以追溯到高德真实候选。